Evaluating Realistic Adversarial Evasion in Machine-Learning-Based Network Intrusion Detection

by Aimukhanbetova Aimzhan

Research Report/Dissertation

Submitted in partial fulfilment for the degree of
Bachelor of Science (BSc) in Computational Sciences with Software Engineering

Supervised by Zhumay Inkar



Heriot-Watt University
School of Mathematical and Computer Sciences

26 November 2026

The copyright in this dissertation is owned by the author. Any quotation from the dissertation or use of any of the information contained in it must acknowledge it as the source of the quotation or information.

Declaration of Authorship

Course code and name	F20PA
Type of assessment	Individual
Document title	Evaluating Realistic Adversarial Evasion in Machine-Learning-Based Network Intrusion Detection
Student name	Aimukhanbetova Aimzhan
Student ID	H00469968

Statement on Generative AI Use:

Generative AI (ChatGPT) was used as a support tool during the project for planning, explaining technical concepts, assisting with Python code and debugging, improving academic English, structuring draft sections, and supporting literature synthesis. Generated material was reviewed and edited by the student. Experimental values reported in this document come from the student's own executed code and project artefacts. References and factual claims are checked against original sources before final submission. The student remains responsible for the accuracy, integrity and originality of the submitted work. A fuller summary is provided in Appendix D.

By signing this form:

- I declare that the work I have submitted for this assessment is my own original work. I have not presented the ideas, words, data, code, designs, analyses, or other intellectual contributions of another person or system as my own.
- My submission is expressed in my own words except where the work of others has been appropriately acknowledged, cited, and referenced. All sources used in the preparation of this assessment, including books, journal articles, websites, software tools, and Generative AI tools, have been properly acknowledged.
- Where Generative AI tools have been used, I have complied with the course's stated policies regarding Generative AI use. I have clearly and accurately disclosed all uses of Generative AI in the assessment, including the specific tools used, the purpose for which they were used, and the extent to which any generated content influenced the final submission.
- I understand that I remain fully responsible for the content, accuracy, integrity, and originality of my submission, regardless of any assistance received from Generative AI or other tools.
- I confirm that I have read, understood and followed the University's Regulations on academic misconduct and am aware of the penalties that may apply if I do not adhere to them.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: Aimukhanbetova Aimzhan **Date:** 26 November 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

Acknowledgements

Abstract

Machine-learning-based network intrusion detection systems (ML-NIDS) can achieve high performance on labelled traffic datasets, yet their reliability under adversarial evasion remains uncertain. A central limitation in much adversarial NIDS research is that perturbations are often applied directly to extracted feature vectors, even though many flow features are derived statistics that a real attacker cannot independently set. This project investigates how measured adversarial vulnerability changes when perturbations are restricted to operationally feasible network actions. CICIDS2017 is used to establish clean-data baselines and to develop a constraint model that distinguishes directly controllable traffic properties from dependent or derived features. Unconstrained feature-space evasion will then be compared with constraint-aware attacks across increasing perturbation budgets, followed by evaluation of selected defences. Preliminary work has produced a cleaned binary dataset containing 2,520,798 flows and a Random Forest pilot model with 99.35% attack recall and a 0.08% false-positive rate under a random stratified split. A small adversarial feasibility prototype has also been implemented, comparing a constraint-aware timing transformation with a simple model-guided unconstrained feature-space search on the same malicious-flow sample. These results demonstrate technical feasibility while motivating stricter source-aware evaluation and more rigorous attack-budget design in the final study.

Table of Contents

Authorship Declaration . . . ii
Acknowledgements . . . iii
Abstract . . . iv
Table of Contents . . . v
1 Introduction . . . 1
1.1 Background and Motivation . . . 1
1.2 Aim and Objectives . . . 2
1.3 Research Questions . . . 2
1.4 Organisation of the Report . . . 2
2 Background Research . . . 3
2.1 Machine Learning for Network Intrusion Detection . . . 3
2.2 Adversarial Evasion and Threat Models . . . 3
2.3 Feature-Space Versus Problem-Space Attacks . . . 4
2.4 Defensive Approaches . . . 4
2.5 Research Gap and Project Positioning . . . 5
3 Methodology and Work Plan . . . 6
3.1 Overall Experimental Design . . . 6
3.2 Dataset and Preprocessing . . . 6
3.3 Baseline Models . . . 7
3.4 Constraint Model . . . 7
3.5 Adversarial Attack Design . . . 8
3.6 Defence Evaluation . . . 9
3.7 Evaluation Metrics and Success Criteria . . . 9
3.8 Second-Semester Work Plan . . . 9
4 Feasibility . . . 10
4.1 Development Environment . . . 10
4.2 Dataset Inspection and Cleaning . . . 10
4.3 Preliminary Random Forest Pilot . . . 11
4.4 Preliminary Adversarial Perturbation Prototype . . . 12
4.5 Feasibility Assessment . . . 13
5 Conclusion . . . 14
References . . . 15
A Project Management . . . 17
A.1 Gantt Chart . . . 17
A.2 Management and Version Control . . . 17
B Risk Analysis and Mitigation . . . 18
C Professional, Legal, Ethical and Social Issues (PLES) . . . 18
D Generative AI Summary . . . 19
E Additional Material . . . 20

1 Introduction

1.1 Background and Motivation

Network intrusion detection systems (NIDS) analyse network activity to identify traffic that may correspond to malicious behaviour. Signature-based systems are effective when attack patterns are already known, but they can be less suitable for previously unseen or rapidly changing attacks. Machine learning (ML) offers a complementary approach in which a detector learns statistical relationships in traffic data and uses them to classify future observations. Recent surveys show that ML and deep-learning approaches have become important components of NIDS research, while also highlighting their vulnerability to adversarial manipulation [1, 2, 3].

Adversarial machine learning studies deliberately constructed inputs that cause a trained model to make an incorrect prediction. The general phenomenon is well established in machine learning [4, 5]. In a NIDS context, the security implication is direct: malicious traffic that is normally detected as an attack may be modified so that a model classifies it as benign. This is an *evasion attack*, because the attacker aims to bypass a detector at inference time while retaining the original malicious objective.

However, network traffic differs from image data, where many common adversarial methods were first developed. A network-flow record is a structured set of values such as flow duration, packet counts, byte counts, packet-length statistics and inter-arrival-time statistics. Many of these values are not independent controls available to an attacker. They are produced by a feature-extraction process from packets and timing behaviour. For example, an attacker may delay packets or add padding, but cannot directly assign an arbitrary value to a derived statistic such as mean packet length or flow bytes per second without changing the underlying traffic that generates it. Surveys of adversarial NIDS research have repeatedly identified this mismatch between mathematical perturbation and operational feasibility as a major open problem [6, 7, 8, 3].

The distinction is commonly described as one between *feature-space* and *problem-space* attacks. In feature space, an adversarial method directly changes the numerical representation supplied to a classifier. This is useful for analysing model sensitivity, but the resulting feature vector may not correspond to traffic that can actually be generated. Problem-space approaches instead modify the underlying artefact or attacker-controlled environment and then observe the resulting features. Catillo et al. demonstrate this distinction by modifying denial-of-service traffic through traffic-control operations rather than editing flow features directly [9]. Verkerken et al. further argue that practical adversarial capability should be defined in terms of the hosts that an attacker actually controls; their systematic review of 316 papers found that many prior studies perturb already collected datapoints rather than the attacker's operational environment [10]. Related work also highlights an inverse feature-mapping problem: a mathematically valid adversarial vector is not necessarily easy, or even possible, to translate back into valid packets [7, 6].

This project therefore does not ask only whether an ML-NIDS can be evaded. Instead, it asks how much of the apparent vulnerability remains when the attack is restricted to modifications that are operationally meaningful. The intended contribution is a controlled comparison between unconstrained feature-space attacks and constraint-aware attacks under the same dataset, classifiers, perturbation budgets and evaluation metrics. The project is defensive: it uses public datasets and offline experimentation to understand and improve detector robustness, not to attack live systems.

1.2 Aim and Objectives

Aim. The aim of this project is to evaluate the adversarial robustness of machine-learning-based network intrusion detection under operationally realistic perturbation constraints, and to compare this robustness with that suggested by unconstrained feature-space attacks.

The project objectives are:

O1. Select and prepare a labelled network-traffic dataset suitable for flow-based intrusion detection and adversarial robustness experiments.

O2. Establish reproducible clean-data baseline performance for at least two machine-learning classifiers.

O3. Construct a constraint model that classifies NIDS features according to attacker controllability, derivation and dependency relationships.

O4. Implement an unconstrained feature-space evasion baseline and a constrained attack that modifies malicious examples only through permitted operations.

O5. Measure attack detection rate as perturbation strength increases, and compare unconstrained and constrained robustness under equivalent evaluation conditions.

O6. Evaluate selected defences, principally adversarial training and feature restriction, with ensembling treated as an extension if time permits.

O7. Quantify the cost of improved robustness using clean-data performance and false-positive rate, rather than reporting adversarial accuracy alone.

1.3 Research Questions

The study is organised around the following research questions:

RQ1. How does constraining adversarial perturbations to operationally feasible network changes affect the measured robustness of ML-based network intrusion detection systems?

RQ2. How does attack detection rate change as the perturbation budget increases for unconstrained and constrained evasion attacks?

RQ3. To what extent can selected defensive techniques recover adversarial robustness, and what cost do they impose on clean-data detection performance and false-positive rate?

These questions are deliberately comparative rather than yes/no questions. The aim is not merely to demonstrate that evasion is possible, but to measure the size and practical meaning of the difference between alternative threat models.

1.4 Organisation of the Report

Section 2 reviews the background literature on ML-based NIDS, adversarial evasion, attack realism and defensive approaches. Section 3 presents the proposed methodology, experimental design and second-semester work plan. Section 4 reports the preliminary dataset preparation and pilot model used to demonstrate feasibility. Section 5 concludes the proposal and summarises the expected contribution. Supporting project-management, risk, PLES and GenAI material is provided after the references.

2 Background Research

2.1 Machine Learning for Network Intrusion Detection

NIDS can be implemented using signatures, statistical anomaly detection, machine learning, or hybrid methods. ML-based approaches are attractive because they can learn non-linear relationships across many traffic features and can be retrained as data evolves. At the same time, performance reported on a benchmark dataset is not equivalent to operational security. A detector can perform extremely well on a clean test set while remaining brittle under deliberate manipulation or distribution shift. This motivates evaluating not only clean predictive performance but also adversarial robustness [1, 11, 3].

This project uses CICIDS2017 as the primary dataset. The dataset was created to include benign activity and multiple attack scenarios, with flow features extracted using CICFlowMeter [12]. It is widely used in NIDS research, which makes it useful for comparison with existing work, but it also has limitations. It is a controlled benchmark captured over specific days and attack scenarios, and its high separability can lead to optimistic results when records are randomly mixed across training and testing. For this reason, the project distinguishes between a random stratified pilot split, used for early feasibility testing, and a stricter source/day-aware evaluation planned for the main experiments.

The initial task is formulated as binary classification: `BENIGN=0` and any malicious label=1. This is appropriate for the primary evasion question. If a DDoS flow is misclassified as PortScan, the attack category is wrong but the intrusion is still detected. A successful evasion for the purpose of this work is specifically a malicious flow being classified as benign. Multiclass analysis may be included as secondary work if time permits, but it is not required for the minimum viable research contribution.

2.2 Adversarial Evasion and Threat Models

Adversarial examples were initially studied extensively in vision and other continuous domains. Goodfellow et al. introduced the Fast Gradient Sign Method (FGSM), showing that small worst-case perturbations can cause confident misclassification [4]. Subsequent work demonstrated transferability and adversarial training at larger scale [5]. These ideas motivated adversarial studies in cybersecurity, but structured network data introduces additional constraints that do not exist in images.

A threat model states what the adversary knows and what the adversary can do. Knowledge may range from white-box access to model architecture and gradients, through grey-box partial knowledge, to black-box interaction. Capability assumptions are equally important. A mathematically powerful attacker that is allowed to alter every feature independently may produce impressive evasion rates, but those results can overstate practical risk if the altered vector cannot be realised as valid network traffic. Apruzzese et al. therefore argue for threat models based on the actual capabilities and circumstances of cyber attackers rather than assumptions inherited from other ML domains [7].

Recent surveys reinforce this concern. He et al. note that directly transplanting adversarial methods from computer vision to NIDS can ignore the NIDS feature and detection pipeline [1]. McCarthy et al. frame functionality preservation as a central requirement in cybersecurity adversarial ML [6]. Alatawi's systematic review organises recent work using a perturbation-realism perspective and highlights the progression from abstract feature-space perturbations toward constraint-aware and executable attacks [8]. Espindola et al. similarly report persistent research biases toward feature-space manipulation and optimistic adversary assumptions [3]. Together these sources motivate treating attack realism as an experimental variable rather than an informal caveat.

2.3 Feature-Space Versus Problem-Space Attacks

A feature-space attack searches for a modified vector x' that changes the classifier decision while remaining close to the original malicious sample x. A generic optimisation can be represented as

$$\text{find } x' \text{ such that } f(x') = \text{BENIGN}, \quad \text{while } d(x, x') \leq \epsilon, \tag{1}$$

where f is the detector, d is a distance or cost function, and ϵ is a perturbation budget. This formulation is convenient because it operates directly on the model input. The limitation is that a small numerical distance does not guarantee semantic or protocol validity.

Network-flow features can be divided conceptually into directly controllable properties, indirectly affected values, derived statistics and practically immutable/contextual fields. For example, an attacker may be able to insert delays, alter packet sizes through padding, split traffic, or change packet scheduling. Those operations then affect multiple derived features simultaneously. In contrast, independently assigning new values to `Flow Bytes/s`, `Packet Length Mean`, `Flow IAT Std` and `Flow Duration` can produce internally inconsistent observations.

Catillo et al. address this issue by applying modifications in the problem space and observing the resulting feature changes, showing that realistic traffic-control perturbations can transfer across multiple ML models [9]. Verkerken et al. formalise *host-space perturbations*, arguing that a real adversary can operate on hosts under their control but cannot normally edit values already observed at a router or NIDS sensor [10]. Yao et al. propose packet-level adversarial traffic generation with a reversible traffic representation and domain constraints in Industrial IoT settings, providing another example of functionality-aware attack construction [13]. Venturi et al. examine problem-space structural attacks against GNN-based NIDS, modelling feasibility constraints when the adversary changes graph structure rather than arbitrary node features [14].

These approaches do not imply that feature-space analysis is useless. Feature-space attacks remain valuable for stress-testing decision boundaries, ranking feature sensitivity and establishing an upper-bound style comparison. The methodological issue is the interpretation of their success. This project therefore treats unconstrained feature-space evasion as a contrast condition and asks how much of its effect remains under a more restrictive operational model.

2.4 Defensive Approaches

Adversarial training is one of the most established robustness techniques. The model is retrained using adversarially modified examples so that its decision boundary becomes less sensitive to the tested perturbations [4, 5]. In NIDS, recent work has explored both conventional and constraint-aware variants. Vitorino et al. benchmark regularly and adversarially trained tree ensembles using constrained adversarial examples on multiple intrusion datasets and report improved robustness without necessarily increasing false alarms [15]. Al Roken et al. propose a hybrid adversarial retraining approach across CICIDS2017 and CSE-CIC-IDS2018, demonstrating that retraining strategy can materially change adversarial performance [16].

A second defence strategy is feature restriction or feature engineering. If robustness depends strongly on features that are easily manipulated, limiting the model to a more stable subset may reduce attack surface, although clean detection accuracy may also fall. This trade-off is particularly relevant to the present project because the constraint table explicitly distinguishes attacker influence over different feature groups. Feature restriction will therefore be evaluated not merely as dimensionality reduction, but as a defence informed

by the threat model.

Ensembling is a further option because attacks that exploit one model's boundary may not transfer perfectly to another. Tafreshian and Zhang combine adversarial training, balancing, feature engineering and ensemble learning in a defensive NIDS framework [17]. DSAAT by Yin et al. is especially relevant because it explicitly considers feature-space and problem-space constraints, including mutability, range and dependency restrictions, while adapting adversarial training across both spaces [18]. Detection-based approaches also exist; for example, Bunzel and Siwakoti detect adversarial evasion against an autoencoder-based NIDS by checking perturbation consistency and reconstruction-error structure [19]. These studies show that there is no single defence objective: robustness must be considered together with clean detection quality, false alarms and computational cost.

Table 1: Selected literature and its role in motivating the project.

Study	**Attack/evaluation space**	**Realism focus**	**Relevance to this project**
Apruzzese et al. [7]	Threat-model / NIDS evaluation	Attacker knowledge and feasible capability assumptions	Motivates explicit modelling of what a real attacker can observe and manipulate
Catillo et al. [9]	Problem space	Traffic-control perturbations applied before feature extraction	Demonstrates that realistic traffic changes can be evaluated without arbitrary independent feature editing
Verkerken et al. [10]	Host space	Perturbations available to an attacker controlling their own host	Supports the distinction between attacker actions and already-observed NIDS feature vectors
Yao et al. [13]	Packet/problem space	Domain-constrained, functionality-aware traffic modification	Informs the use of packet/timing actions and dependency-preserving transformations
Vitorino et al. [15]	Constrained adversarial benchmark	Robustness of enterprise NIDS models under constrained examples	Supports matched robustness comparison and defence evaluation
Yin et al. [18]	Dual feature/problem space	Mutability, range and dependency constraints in adversarial training	Directly motivates the planned constraint categories and constraint-aware defence evaluation

2.5 Research Gap and Project Positioning

The literature establishes three points. First, ML-based NIDS can be highly accurate on benchmark data. Second, these models can be vulnerable to adversarial manipulation. Third, the practical meaning of an adversarial result depends strongly on where and how the perturbation is applied. Recent surveys and problem-space studies agree that operational feasibility, feature dependencies and functionality preservation require more explicit treatment [6, 8, 3, 10].

The intended contribution of this project is therefore not to claim the first realistic adversarial NIDS attack. Instead, the project will build a reproducible comparison in which unconstrained and constrained evasion are evaluated on the same cleaned dataset, same baseline models, same binary objective and same detection metrics. This isolates the effect of the realism constraint itself. The follow-on defence experiments then ask whether robustness can be recovered and at what false-positive cost. For a final-year project, this controlled comparative design provides a focused and measurable contribution while remaining feasible within the available time.

3 Methodology and Work Plan

3.1 Overall Experimental Design

The proposed pipeline consists of five stages: dataset preparation, clean baseline modelling, constraint modelling, adversarial evaluation, and defence evaluation. Figure 1 summarises the planned end-to-end workflow and distinguishes the preparation already demonstrated from the Semester 2 experimental stages.

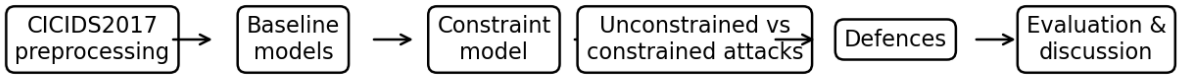


Figure 1: Overall experimental workflow. Dataset preparation and the pilot baseline have already been exercised; the constraint-aware attack, defence and final evaluation stages are planned for Semester 2.

Table 2: Project status at the time of D1 preparation.

Work package	**Status in D1**	**Role after D1**
Literature review and research gap	In progress / sufficient for proposal	Continue targeted reading as constraints and defence choices are finalised
CICIDS2017 inspection and cleaning	Completed	Reuse the frozen preprocessing pipeline for final experiments
Binary Random Forest pilot	Completed	Replace/extend with final random and source-aware baseline evaluations
Constraint model	Preliminary design + timing prototype	Finalise evidence, dependencies and extend transformation functions
Unconstrained and constrained evasion	Preliminary prototype completed	Replace exploratory prototypes with rigorous final attacks and run perturbation-budget experiments
Defence evaluation	Planned	Evaluate at least one core defence and its clean-data cost
Final evaluation and discussion	Planned	Complete for D3 using measured attack and defence results

3.2 Dataset and Preprocessing

CICIDS2017 is the primary dataset because it contains labelled benign and malicious traffic together with flow-level numerical features suitable for tabular classifiers [12]. The eight MachineLearningCSV files have already been downloaded and inspected. Preprocessing is designed to be reproducible and to avoid changing the raw source files.

The implemented preprocessing procedure is:

1. normalise column names by removing leading and trailing whitespace;

2. preserve a `SourceFile` field for later day/source-aware evaluation, but exclude it from model inputs;
3. remove `Fwd Header Length.1` after verifying that it is identical to `Fwd Header Length` in all eight files;
4. convert positive and negative infinity to missing values and remove rows containing invalid numerical values;
5. merge the source files and remove exact global duplicate observations before any train/test split;
6. create a binary target where benign traffic is 0 and all attacks are 1.

Duplicate removal is performed before splitting because identical records in both training and testing could inflate measured performance. The raw attack label and `SourceFile` remain available for analysis, but neither is supplied to the classifier. This avoids obvious label leakage and allows the final evaluation to investigate whether results depend on capture source.

3.3 Baseline Models

Random Forest is the first baseline because it is well suited to large tabular datasets, can model non-linear relationships and has already been used successfully in NIDS research. The pilot configuration uses 100 trees, a fixed random state, all available CPU cores and balanced class weights. A second baseline will be selected from a strong tree-ensemble alternative such as XGBoost, subject to runtime and supervisor-approved scope. The objective is not to search exhaustively for the highest clean accuracy, but to establish at least two credible detectors with different decision mechanisms.

Two evaluation regimes are planned. The first is a stratified random split, useful for comparability and rapid experimentation. The second is a source/day-aware split using the retained `SourceFile` information. The stricter regime is important because random mixing may allow very similar patterns from one capture scenario to appear in both training and testing. Final conclusions will therefore distinguish within-distribution random-split performance from source-aware generalisation.

3.4 Constraint Model

The constraint table is the central design artefact of the project. Each feature will be assigned to one of four conceptual categories:

1. **directly controllable**: a property the attacker can intentionally change through an operational action;
2. **indirectly affected**: a property that changes as a consequence of an attacker action but is not independently set;
3. **derived/dependent**: a statistic computed from other packet or timing properties and therefore subject to consistency relations;
4. **immutable or context-limited**: a property that the assumed attacker cannot freely change without altering the attack scenario or target service.

For each relevant feature, the table will record the feasible action, direction/range restrictions, dependent features that must be recomputed, and evidence supporting the constraint. Ranges will not be chosen

arbitrarily; they will be based on published problem-space work, protocol semantics and controlled sensitivity experiments. Table 3 shows the intended structure rather than final constraint values.

Table 3: Draft structure of the feature-constraint table. Final constraints will be evidence-based.

Feature / group	**Control type**	**Attacker action**	**Dependency consideration**
Flow duration / IAT	Indirect	Timing jitter / delay	Rate and IAT statistics must change consistently
Packet lengths	Partial/direct	Padding or packetisation changes	Mean, min, max, variance and byte totals are coupled
Flow Bytes/s	Derived	No direct assignment	Recompute from byte totals and duration
Packet counts	Indirect	Fragmentation / packet insertion where valid	Subflow and rate features may change
Destination port	Context-limited	Service/target choice	May change attack scenario rather than merely perturb it

3.5 Adversarial Attack Design

A preliminary adversarial feasibility prototype has already been implemented to exercise this comparison pipeline before D1 submission. The constraint-aware branch applies a timing-oriented transformation in which duration and inter-arrival-time features are scaled together while rate-derived features are adjusted inversely. The unconstrained branch uses a simple model-guided feature-space search that generates several independently perturbed candidates and retains the candidate with the lowest Random Forest ATTACK probability. These prototypes are exploratory rather than final attacks, but they confirm that the full evaluation loop can be executed end-to-end.

The unconstrained experiment will provide a feature-space reference condition. Its role is to quantify model sensitivity when the adversary is allowed to modify a selected numerical feature set within a defined budget. Because Random Forest is non-differentiable, the exact optimisation method will be chosen to suit tree models, for example constrained search, surrogate-based optimisation, or another model-agnostic method. Gradient-based attacks may be used only for differentiable comparison models and will not be forced onto Random Forest simply because they are common in vision research.

The constrained attack will operate through transformation functions that represent attacker actions rather than arbitrary independent feature edits. A timing transformation, for example, will modify timing-relevant primitive values and recompute dependent rate and inter-arrival statistics. A packet-size or padding transformation will similarly update byte totals and packet-length statistics consistently. Each generated adversarial example must satisfy three conditions: it originates from a malicious sample, it remains inside defined validity constraints, and the malicious objective is assumed to remain operational under the chosen perturbation model.

Perturbation strength will be evaluated at multiple budgets. The exact scale will depend on the constraint type, so a normalised cost measure may be used to compare different actions. The primary output will be detection rate versus perturbation budget for unconstrained and constrained attacks. Attack success rate will be reported only for originally detected malicious samples so that failure of the clean baseline is not incorrectly counted as adversarial success.

3.6 Defence Evaluation

The minimum defence scope contains two techniques:

1. **adversarial training**: retrain models with constrained adversarial examples generated from the training data;
2. **feature restriction**: retrain models using a reduced feature set that excludes or reduces dependence on highly manipulable features.

An ensemble is a stretch objective if the core experiments are completed on schedule. This prioritisation prevents defence breadth from displacing the main research contribution, which is the constrained-versus-unconstrained comparison.

Every defence will be evaluated on both clean and adversarial data. A defence will not be considered successful merely because adversarial detection improves. Its false-positive rate, clean attack recall and F1-score must also remain acceptable. This explicitly measures the robustness–utility trade-off identified in previous defence work [15, 17, 18].

3.7 Evaluation Metrics and Success Criteria

The primary metrics are:

- **Attack recall / detection rate**: proportion of malicious observations correctly detected;
- **False-positive rate (FPR)**: proportion of benign observations incorrectly labelled as attacks;
- **Precision**: proportion of attack predictions that are correct;
- **F1-score**: harmonic mean of precision and recall;
- **Attack success rate**: proportion of originally detected malicious samples converted to benign predictions by the adversarial procedure.

A successful final project must, at minimum, (i) reproduce a clean baseline, (ii) implement and document the constraint model, (iii) generate a matched unconstrained/constrained robustness comparison over more than one perturbation budget, and (iv) evaluate at least one defence including its clean-data cost. A second model and second defence strengthen the study but are secondary to completing the core comparative experiment correctly.

3.8 Second-Semester Work Plan

The main implementation work is intentionally staged after D1/D2. The first-semester deliverables establish feasibility; the second semester executes the full adversarial study. The planned order is:

1. finalise literature review and evidence-based feature constraints;
2. establish final Random Forest and second-model baselines under random and source-aware splits;
3. refine the exploratory unconstrained prototype into the final reference attack;

4. extend the preliminary timing transformation into evidence-based constrained traffic transformations with dependency recomputation;
5. run perturbation-budget experiments and analyse attack validity;
6. implement adversarial training and feature-restriction defences;
7. complete statistical/graphical evaluation, limitations and discussion;
8. write, revise and submit D3; prepare D5 demonstration and Q&A.

4 Feasibility

4.1 Development Environment

The project is implemented in Python using pandas and NumPy for data processing and scikit-learn for baseline modelling and metrics. Jupyter notebooks in Visual Studio Code are used for exploratory analysis and reproducible experiments. GitHub is used for private version control and project management. The repository is structured into planning, research, design, implementation, testing, meetings and administration folders, with a project logbook and literature log maintained throughout the year. Large raw datasets and trained-model artefacts are excluded from version control.

4.2 Dataset Inspection and Cleaning

All eight CICIDS2017 MachineLearningCSV files were inspected. The raw combined dataset contained 2,830,743 flow records and 79 original columns (78 input features plus `Label`). The label distribution was strongly imbalanced, with benign traffic dominating the dataset and several attack classes represented by very few observations.

Data-quality checks identified missing values, infinite values, within-file duplicates and a redundant duplicate feature. Missing values occurred in `Flow Bytes/s`; infinite values occurred in `Flow Bytes/s` and `Flow Packets/s`. Across all files, 2,867 rows contained at least one missing or infinite value. The columns `Fwd Header Length` and `Fwd Header Length.1` were compared programmatically and were identical in all eight source files, so the latter was removed.

After invalid numerical rows were removed, a global duplicate check was performed across the merged dataset. This identified 307,078 exact duplicate observations. The duplicates were not evenly distributed across attack labels, which confirmed the need to inspect them before removal. Exact duplicates were then removed before any train/test split to reduce the risk of identical observations appearing in both sets.

The final cleaned dataset contains 2,520,798 flows. Of these, 2,095,057 (83.11%) are benign and 425,741 (16.89%) are malicious under the binary target. Verification confirmed zero remaining missing values, zero infinite values and zero remaining exact duplicates. The cleaned dataset is saved separately, leaving the raw CSV files unchanged.

Table 4: Summary of completed CICIDS2017 preprocessing.

Item	Result
Raw flows	2,830,743
Rows removed for NaN/Infinity	2,867
Exact global duplicates removed	307,078
Final clean flows	2,520,798
Benign flows	2,095,057 (83.11%)
Attack flows	425,741 (16.89%)
Remaining NaN / Infinity / exact duplicates	0 / 0 / 0

4.3 Preliminary Random Forest Pilot

A pilot Random Forest was trained to test the end-to-end ML pipeline. A random sample of 300,000 cleaned flows was selected with a fixed random seed. This sample was divided into 240,000 training observations and 60,000 test observations using an 80/20 stratified split so that the benign/attack ratio was preserved in both sets. The model used 77 ML features; `Label`, `BinaryLabel` and `SourceFile` were excluded from the input matrix.

The pilot achieved 0.9982 accuracy, 0.9959 precision, 0.9935 attack recall/detection rate, 0.9947 F1-score and a 0.0008 false-positive rate. Its confusion matrix contained 49,777 true negatives, 42 false positives, 66 false negatives and 10,115 true positives. These values are project results from the executed notebook, not literature values.

Table 5: Preliminary Random Forest pilot results.

Metric	Pilot result
Accuracy	0.9982
Precision	0.9959
Attack recall / detection rate	0.9935
F1-score	0.9947
False-positive rate	0.0008

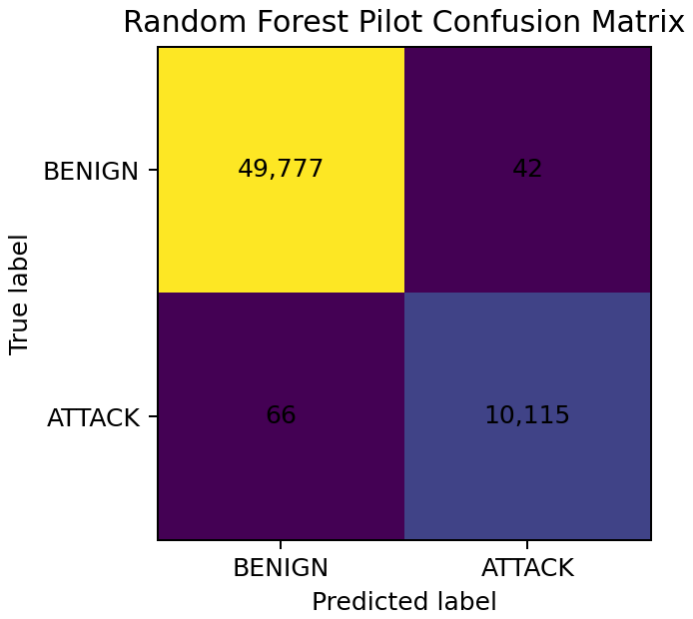


Figure 2: Confusion matrix for the 60,000-flow Random Forest pilot test set. The pilot is feasibility evidence and not the final estimate of generalisation.

The very high result is evidence that the software pipeline is functioning and that a strong clean baseline can be established. It is not treated as a final estimate of real-world generalisation. Because the pilot uses a random stratified split, similar traffic patterns from the same capture scenarios may be represented in both training and testing. The full study will therefore include source/day-aware evaluation and will interpret any difference between the two regimes as part of the model-validity analysis.

4.4 Preliminary Adversarial Perturbation Prototype

To test whether the planned adversarial evaluation pipeline was technically viable, a small proof-of-concept experiment was implemented using malicious flows that the pilot Random Forest initially classified correctly. A fixed sample of 5,000 such flows was used for the direct comparison so that both prototype branches were evaluated on the same observations.

The **constraint-aware timing** branch applied a simple dependency-preserving transformation: timing-related features such as flow duration and inter-arrival-time statistics were scaled upward by a common factor, while rate-derived features such as packets per second and bytes per second were adjusted inversely. This is still a feature-level prototype rather than a packet-level problem-space attack, but it deliberately avoids changing those related values independently.

The **unconstrained** branch used a model-guided random search over nine selected numerical features. For each malicious flow, multiple independently perturbed candidates were generated and the candidate with the lowest Random Forest ATTACK probability was retained. This provides an adversarial objective, but it remains an exploratory feature-space search and does not enforce semantic dependencies between modified features.

Table 6 summarises the same-sample comparison across nominal perturbation budgets. At the largest nominal budget (30%), the constraint-aware timing prototype produced an evasion rate of 1.06%, while the unconstrained search produced an evasion rate of 0.30%. The unconstrained search nevertheless reduced the mean Random Forest ATTACK probability from 0.9958 at baseline to 0.9278 at the 30% setting,

showing that it moved samples toward the benign side of the model even when most predictions did not cross the decision boundary.

Table 6: Preliminary same-sample robustness comparison on 5,000 initially detected malicious flows.

Budget	Constrained DR	Unconstrained DR	Constrained ER	Unconstrained ER
0%	1.0000	1.0000	0.0000	0.0000
5%	0.9950	0.9994	0.0050	0.0006
10%	0.9916	0.9992	0.0084	0.0008
20%	0.9904	0.9978	0.0096	0.0022
30%	0.9894	0.9970	0.0106	0.0030

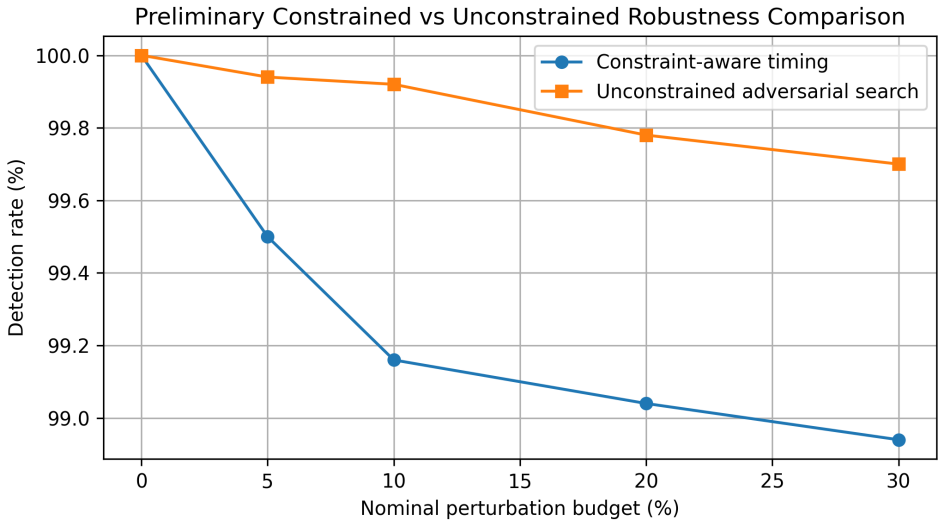


Figure 3: Preliminary constrained versus unconstrained robustness comparison. The x-axis is a nominal percentage budget used for this feasibility experiment; the two prototype budgets are not yet physically equivalent and the result is therefore exploratory rather than a final robustness comparison.

The result should not be interpreted as evidence that constrained attacks are generally stronger than unconstrained attacks. The prototypes modify different feature sets and use different search procedures, and the shared percentage scale is only a nominal comparison. Instead, the experiment provides feasibility evidence that the project can (i) select originally detected malicious flows, (ii) apply both dependency-aware and unconstrained transformations, (iii) re-evaluate them with the same detector, and (iv) measure detection and evasion consistently. A preliminary impurity-based Random Forest feature-importance check also indicated that packet-length and traffic-volume variables dominated the highest-ranked features, which motivates broader packet-size and padding-related constraints in the final implementation.

4.5 Feasibility Assessment

The preliminary work demonstrates that the core project is achievable with the available tools and hardware. The full CICIDS2017 dataset can be processed locally; a reproducible cleaning pipeline has been implemented; a 77-feature binary classification pipeline trains successfully; the required NIDS metrics have been computed; and both constrained and unconstrained perturbation prototypes can be generated and re-evaluated automatically. The remaining work is conceptually harder but builds on functioning

infrastructure rather than requiring a new platform.

The largest technical uncertainty is not whether a classifier can be trained, but how to construct sufficiently defensible constraints and maintain feature consistency during adversarial transformation. This risk is manageable because the literature already provides problem-space and host-space models that can be used to justify the constraint categories [9, 10, 13]. The minimum viable implementation can also focus on a smaller set of well-supported traffic actions if modelling all 77 features proves unnecessary. This fallback preserves the research question while reducing implementation risk.

5 Conclusion

This proposal addresses a specific limitation in adversarial evaluation of ML-based network intrusion detection: a numerically successful feature-space perturbation is not necessarily an action available to a real network attacker. Prior research increasingly recognises the importance of functionality preservation, realistic threat models, host-space capability and feature dependencies [6, 7, 9, 10, 3]. The project therefore proposes a controlled comparison between unconstrained feature-space evasion and operationally constrained evasion rather than treating all adversarial examples as equally realistic.

Substantial preparation has already been completed. CICIDS2017 has been inspected and cleaned from 2,830,743 raw flows to 2,520,798 unique valid flows, with the preprocessing decisions documented and verified. A preliminary Random Forest pipeline has also been executed successfully, achieving 99.35% attack recall with a 0.08% false-positive rate on a 300,000-flow pilot sample. In addition, a same-sample adversarial feasibility prototype compared a dependency-aware timing transformation with a simple model-guided unconstrained search over nominal budgets up to 30%. The timing prototype reached a 1.06% evasion rate at the largest setting, while the unconstrained search reached 0.30%; these exploratory figures are not treated as final attack comparisons but confirm that the adversarial evaluation pipeline is functioning. Together, these results provide concrete feasibility evidence while motivating a stricter final evaluation design.

The second-semester work will finalise the evidence-based constraint table, establish final baselines, replace the exploratory prototypes with matched final unconstrained and constrained attacks, evaluate robustness over rigorously defined perturbation budgets, and test selected defences. The central expected contribution is a clearer distinction between apparent model vulnerability in feature space and vulnerability under feasible attacker actions. This distinction is intended to support more cautious interpretation of adversarial NIDS results and more defensible evaluation of robustness.

References

[1] K. He, D. D. Kim, and M. R. Asghar. *Adversarial Machine Learning for Network Intrusion Detection Systems: A Comprehensive Survey*. IEEE Communications Surveys & Tutorials, 25(1):538–566, 2023. DOI: https://doi.org/10.1109/COMST.2022.3233793.

[2] S. Ennaji, F. De Gaspari, D. Hitaj, A. Kbidi, and L. V. Mancini. *Adversarial Challenges in Network Intrusion Detection Systems: Research Insights and Future Prospects*. IEEE Access, 13:148613–148645, 2025. DOI: https://doi.org/10.1109/ACCESS.2025.3600984.

[3] A. da S. Espindola, A. O. Santin, A. Casimiro, P. M. Ferreira, and E. K. Viegas. *Understanding the Adversary: A Survey of Adversarial Machine Learning in Network Intrusion Detection*. Computer Science Review, 62:100995, 2026. DOI: https://doi.org/10.1016/j.cosrev.2026.100995.

[4] I. J. Goodfellow, J. Shlens, and C. Szegedy. *Explaining and Harnessing Adversarial Examples*. International Conference on Learning Representations (ICLR), 2015. https://arxiv.org/abs/1412.6572.

[5] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial Machine Learning at Scale*. arXiv:1611.01236, 2017. https://arxiv.org/abs/1611.01236.

[6] A. McCarthy, E. Ghadafi, P. Andriotis, and P. Legg. *Functionality-Preserving Adversarial Machine Learning for Robust Classification in Cybersecurity and Intrusion Detection Domains: A Survey*. Journal of Cybersecurity and Privacy, 2(1):154–190, 2022. DOI: https://doi.org/10.3390/jcp2010010.

[7] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni. *Modeling Realistic Adversarial Attacks against Network Intrusion Detection Systems*. arXiv:2106.09380, 2021. https://arxiv.org/abs/2106.09380.

[8] H. A. Alatawi. *Adversarial Evasion in Machine-Learning-Based Network Intrusion Detection: A Systematic Review, Threat Modeling, and Research Roadmap*. IEEE Access, 14:122143–122176, 2026. DOI: https://doi.org/10.1109/ACCESS.2026.3720554.

[9] M. Catillo, A. Pecchia, A. Repola, and U. Villano. *Towards Realistic Problem-Space Adversarial Attacks against Machine Learning in Network Intrusion Detection*. Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES), 2024. DOI: https://doi.org/10.1145/3664476.3669974.

[10] M. Verkerken, L. D'hooge, B. Volckaert, F. De Turck, and G. Apruzzese. *"What is the Problem Space?" Defining Host-space Adversarial Perturbations against Network Intrusion Detection Systems*. Proceedings of the 21st ACM Asia Conference on Computer and Communications Security (ASIA CCS), pp. 1043–1059, 2026. DOI: https://doi.org/10.1145/3779208.3807482.

[11] M. Raj, N. D. Bastian, L. Fiondella, and G. Kul. *Categorical Robustness Assessment and Model Evaluation for Machine Learning Based Network Intrusion Detection Systems*. arXiv:2606.12075, 2026. https://arxiv.org/abs/2606.12075.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116, 2018. DOI: https://doi.org/10.5220/0006639801080116.

[13] W. Yao, H. Peng, Q. Li, and X. Shen. *Modeling Realistic Adversarial Traffic Against Deep Learning-Based Intrusion Detection System in Industrial IoT*. IEEE Internet of Things Journal, 12:29540–29554, 2025.

[14] A. Venturi, D. Stabili, and M. Marchetti. *Problem Space Structural Adversarial Attacks for Network Intrusion Detection Systems Based on Graph Neural Networks*. arXiv:2403.11830, 2024. https://arxiv.org/abs/2403.11830.

[15] J. Vitorino, M. Silva, E. Maia, and I. Praca. *An Adversarial Robustness Benchmark for Enterprise Network Intrusion Detection*. arXiv:2402.16912, 2024. https://arxiv.org/abs/2402.16912.

[16] N. Al Roken, H. Hacid, A. Bouridane, and A. Hussain. *Hybrid Adversarial Retraining Approach for a Proactive Network Intrusion Detection System*. IEEE Open Journal of the Communications Society, 2026. DOI: https://doi.org/10.1109/OJCOMS.2026.3694298.

[17] B. Tafreshian and S. Zhang. *A Defensive Framework Against Adversarial Attacks on Machine Learning-Based Network Intrusion Detection Systems*. 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2024. DOI: https://doi.org/10.1109/TrustCom63139.2024.00337.

[18] S. Yin, J. Cai, and Y. Huang. *DSAAT: Dual-Space Adaptive Adversarial Training for Robust Network Intrusion Detection*. 2026 11th International Conference on Computer and Communication Systems (ICCCS), pp. 1010–1017, 2026. DOI: https://doi.org/10.1109/ICCCS69761.2026.11613290.

[19] N. Bunzel and A. Siwakoti. *Detecting Adversarial Evasion Attacks Against Autoencoder-Based Network Intrusion Detection Systems*. arXiv:2607.01194, 2026. https://arxiv.org/abs/2607.01194.

A Project Management

A.1 Gantt Chart

The Gantt chart below summarises the planned project schedule from October 2026 to April 2027. It separates Semester 1 preparation and feasibility work from the larger Semester 2 implementation and evaluation phase, while D4 project management and the logbook continue throughout the project.

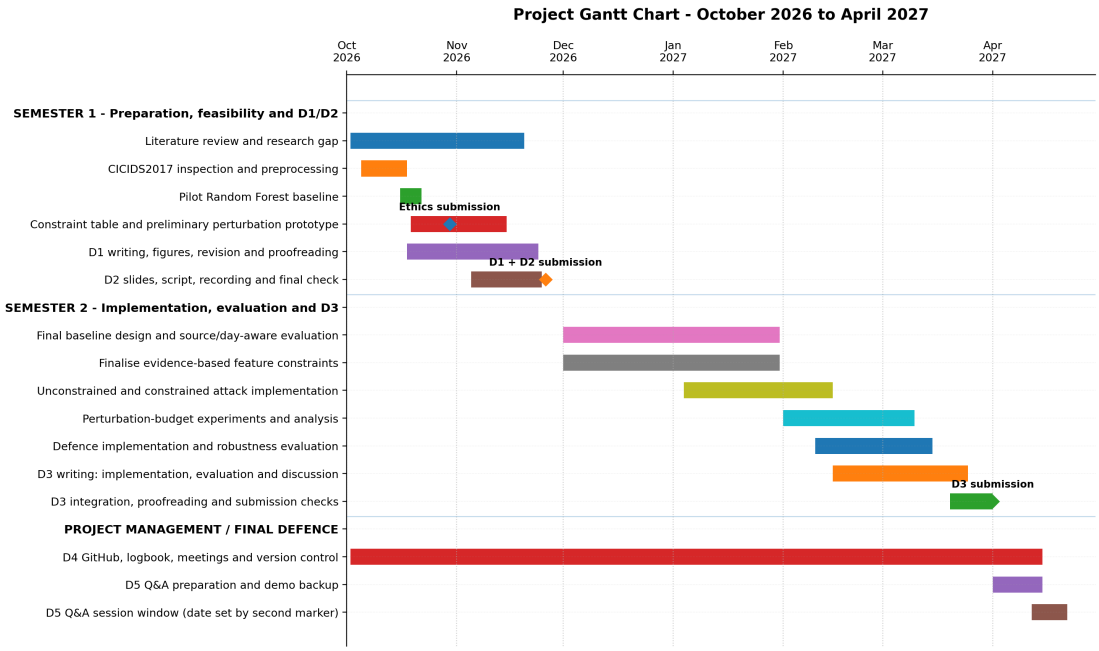


Figure 4: Gantt chart for the project from October 2026 to April 2027. Red diamond markers indicate fixed submission milestones. D4 project management continues throughout the project, while the D5 Q&A date will be scheduled by the second marker.

A.2 Management and Version Control

The project is managed through a private GitHub repository. The repository contains separate folders for planning, research, design, implementation, testing, meetings and administration, together with a `LOGBOOK.md` journal and literature log. Commits are used to record meaningful increments such as dataset inspection, preprocessing changes, baseline experiments and report drafts. Raw datasets and large generated artefacts are excluded from Git using `.gitignore`. Meeting notes and decisions are recorded so that changes to scope can be traced.

The project follows a lightweight iterative approach rather than a formal Scrum implementation. Work is divided into short technical and writing tasks with explicit dependencies: literature supports the constraint model; preprocessing supports baselines; baselines support attack evaluation; and attack results support defence selection. D4 therefore runs continuously alongside the technical work instead of being postponed until the end of the project.

B Risk Analysis and Mitigation

Risk	Likelihood	Impact	Mitigation	Fallback / contingency
Constraint model becomes too broad	Medium	High	Prioritise a small set of well-supported attacker actions and document dependencies explicitly	Restrict final attack to timing and packet-size transformations while preserving the comparative research question
Attack implementation is incompatible with a model	Medium	Medium	Use model-appropriate attacks; avoid forcing gradient methods onto non-differentiable RF	Use surrogate/model-agnostic optimisation for RF and retain differentiable attack only for second model
Random split overestimates performance	High	Medium	Preserve source metadata and add source/day-aware evaluation	Report both regimes and frame random split as comparability/pilot only
Computational limits	Medium	Medium	Pilot on subsets, profile memory, use efficient tree models and incremental experiment design	Reduce model count/budgets while keeping at least one complete baseline–attack–defence chain
Defence scope becomes excessive	Medium	High	Treat adversarial training and feature restriction as core; ensemble as stretch goal	Drop ensemble and focus analysis on two defences
Literature constraints are insufficiently justified	Medium	High	Maintain literature log and require a source/protocol rationale for every constraint	Mark unsupported features as non-perturbable rather than inventing ranges
Code or data loss	Low	High	Private Git version control, separate raw/processed data, regular backup	Reconstruct processed data from documented preprocessing notebook
Schedule pressure near D3	Medium	High	Finish core experiments before extensive optional work; write methods as implementation stabilises	Freeze scope and prioritise evaluation/discussion over additional attack variants

C Professional, Legal, Ethical and Social Issues (PLES)

Professional

The project should follow professional principles of reproducibility, accurate reporting and proportionate security research. Clean and adversarial results will be reported with limitations, and unusually high benchmark performance will not be presented as evidence of real-world deployment readiness. Version

control and a project log provide an auditable development history. Claims about previous work will be supported by primary papers where possible rather than relying on uncited summaries.

Legal

The project uses public research datasets and does not target live networks. Dataset licences and publication terms will be respected, and copyrighted papers will be cited rather than redistributed. No unauthorised access, interception or modification of third-party systems is required. If additional tools or datasets are introduced, their licences and permitted research use will be checked before inclusion.

Ethical

Ethics application status: [UPDATE].

The work is defensive and evaluates evasion offline using labelled datasets. Nevertheless, adversarial methods are dual-use because the same ideas could inform evasion of deployed detectors. The dissertation will therefore focus on experimental methodology, constraints and defensive conclusions rather than operational instructions for attacking real organisations. No human participants or personal data collection is planned. The University's ethics process will be followed for the use of secondary data and any subsequent methodological changes.

Social

More robust NIDS may reduce missed attacks and improve confidence in automated security monitoring, but excessive false positives can waste analyst time and encourage alert fatigue. For that reason, false-positive rate is treated as a first-class metric when evaluating defences. A further social issue is overconfidence in benchmark accuracy: presenting unrealistic adversarial results or unrealistically strong clean results without context can distort expectations about security. The project therefore aims to distinguish benchmark sensitivity from operationally plausible risk.

D Generative AI Summary

Generative AI has been used as a support tool during the project. Uses include explaining unfamiliar technical concepts, helping structure the research plan, suggesting code for data inspection and baseline experiments, debugging Python errors, language editing, drafting report sections, and helping organise literature into themes. The project author executed the code, supplied the resulting outputs, made project decisions, and is responsible for validating all numerical values and technical interpretations.

For literature-related writing, generated text is not treated as a source. Claims are checked against the original paper or a verified bibliographic record, and the paper itself is cited. Where a source has not yet been verified, the final report must not present a fabricated citation. AI-generated draft prose is reviewed and edited by the author before submission, and the use of GenAI is declared in accordance with the assessment requirements.

Representative prompt categories include: "explain the difference between feature-space and problem-space adversarial attacks in simple terms", "help interpret these CICIDS2017 preprocessing outputs", "draft a methodology section using only the supplied papers and verified results", and "debug this Python

error". The generated outputs contributed to planning and drafting but did not replace the author's responsibility for experimental execution, evidence checking, interpretation or final authorship.

E Additional Material

Possible supporting material includes selected notebook screenshots, the full feature-constraint table, supplementary class-distribution tables, extended pilot metrics, and code listings where they aid reproducibility but are not essential to understand the main body. No attack/defence result figures are included in D1 because those experiments are planned for Semester 2; they will be added to D3 only after the corresponding code has been executed and the results have been verified.